

Programação Orientada a Objetos - CST em Análise e Desenvolvimento de Sistemas

APIs Java: Enum e Tipos genéricos

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Enumerações em Java: o tipo Enum

Motivação

- Criar uma classe para representar uma disciplina com os seguintes atributos:
 - Sigla da disciplina
 - Dias da semana em que a disciplina é ministrada
- Para representar os dias da semana, podemos usar uma coleção de Strings ou inteiros
 - Exemplo: Segunda, Terça, Quarta, etc
 - Exemplo: 0 para Domingo, 1 para Segunda, etc
- Problemas:
 - Erros de digitação: Segunda, Segunda-feira, etc
 - Falta de clareza: o que significa o número 1? Segunda ou Terça?

Classe Disciplina

Constantes podem deixar o código mais claro, mas não evitam erros de digitação

```
public class Disciplina{  
    public static final String SEGUNDA = "Segunda";  
    public static final String TERCA   = "Terça";  
    public static final String QUARTA  = "Quarta";  
    public static final String QUINTA  = "Quinta";  
    public static final String SEXTA   = "Sexta";  
  
    private String sigla;  
    private String diasDaSemana[];  
  
    public Disciplina(String sigla, String diasDaSemana[]){  
        this.sigla = sigla;  
        this.diasDaSemana = diasDaSemana;  
    }  
}
```

```
public class App{  
  
    public static void main(String[] args) {  
  
        String[] aulas = {Disciplina.TERCA, Disciplina.SEXTA};  
        Disciplina poo = new Disciplina("POO", aulas);  
  
        // Qualquer string pode ser usada, mesmo que não seja  
        // um dia da semana válido  
        String[] errado = {"Joao", "Maria"};  
        Disciplina ruim = new Disciplina("POO", errado);  
    }  
}
```

Enumerações

O tipo Enum em Java

- Enumerações são tipos de dados que consistem em um conjunto fixo de constantes
 - Podem possuir atributos, métodos e construtores
- Vantagens:
 - Código mais legível e fácil de entender
 - Erros de digitação são evitados, pois o compilador verifica os valores
 - Permite associar métodos e atributos a cada constante
- Em Java, o tipo Enum é uma classe especial que representa um conjunto de constantes nomeadas
 - Seus atributos devem ser públicos, estáticos e finais

```
public enum DiasDaSemana{  
    DOMINGO,  
    SEGUNDA,  
    TERCA,  
    QUARTA,  
    QUINTA,  
    SEXTA,  
    SABADO  
}
```

Enum para dias da semana

- As constantes do Enum são instâncias da própria classe Enum
- Por convenção, os nomes das constantes são escritos em letras maiúsculas

```
public enum DiasDaSemana{  
    DOMINGO,  
    SEGUNDA,  
    TERCA,  
    QUARTA,  
    QUINTA,  
    SEXTA,  
    SABADO  
}
```

```
public class App{  
  
    public static void main(String[] args) {  
  
        DiasDaSemana aulas[] = {DiasDaSemana.TERCA, DiasDaSemana.SEXTA};  
        Disciplina poo = new Disciplina("POO", aulas);  
  
    }  
}
```

Enum com atributos e métodos

```
public enum Cor{  
    PRETO(0), VERMELHO(1), VERDE(2), AMARELO(3),  
    AZUL(4), ROXO(5), CIANO(6), BRANCO(7);  
  
    public final int codigo;  
  
    Cor(int c){  
        this.codigo = c;  
    };  
  
    public static Cor getByCodigo(int c){  
        for (Cor cor: Cor.values()){  
            if (c == cor.codigo){  
                return cor;  
            }  
        }  
        throw new IllegalArgumentException("código inválido");  
    }  
}
```

«enumeration» Cor
PRETO VERMELHO VERDE AMARELO AZUL ROXO CIANO BRANCO
+codigo: int
+getByCodigo(int c): Cor

Uso do Enum em switch

```
public class App{

    public static void main(String[] args) {
        Cor corDoFundo = Cor.getByCodigo(0);
        Cor corDoTexto = Cor.getByCodigo(7);

        System.out.println("Cor do fundo: " + corDoFundo);

        switch(corDoFundo){
            case PRETO -> {
                switch(corDoTexto){
                    case PRETO -> System.out.println("não me parece uma boa combinação");
                    case BRANCO -> System.out.println("me parece uma boa combinação");
                }
            }
        }
    }
}
```

Exercício 1: Sistema solar

- Crie um Enum para representar os planetas do sistema solar
- Cada planeta deve armazenar sua posição em relação ao Sol (exemplo: Terra = posição 3)
- Implemente um método para retornar o planeta correspondente a uma posição dada

Exercício 2: Baralho francês

- Um baralho tem 52 cartas: 13 valores para cada um dos 4 naipes (paus, ouros, copas, espadas)
- Valores das cartas: Ás, 2-10, Valete (J), Dama (Q), Rei (K)
- Implemente um conjunto de classes para representar um baralho de cartas
 - Implemente uma classe Carta com atributos para valor e naipe
 - Use uma coleção para armazenar as cartas do baralho



Tipos genéricos em Java

Motivação

Modele uma classe **Caixa** capaz de armazenar um objeto de qualquer tipo Java

- Ao usar `Object`, podemos armazenar qualquer tipo de objeto, mas perdemos a verificação de tipo em tempo de compilação
- Ao recuperar o objeto, é necessário fazer *typecasting*, o que pode levar a erros em tempo de execução

```
public class Caixa{  
  
    private Object conteudo;  
  
    public Caixa(Object obj){  
        this.conteudo = obj;  
    }  
  
    public void set(Object obj){  
        this.conteudo = obj;  
    }  
  
    public Object getConteudo(){  
        return this.conteudo;  
    }  
}
```

Problemas de usar Object para armazenar qualquer tipo

```
public class App{
    public static void main(String[] args){
        String s = "Olá mundo";
        Pessoa p = new Pessoa("Joao");

        Caixa c = new Caixa(s);
        // typecasting obrigatório
        String outra = (String) c.getConteudo();

        Caixa d = new Caixa(p);

        // erro em tempo de execução, pois o conteúdo é um objeto Pessoa
        String errado = (String) d.getConteudo();
    }
}
```

- É mais fácil detectar *bugs* em tempo de compilação do que em tempo de execução

Tipos genéricos em Java

<https://docs.oracle.com/javase/tutorial/java/generics>

- Tipos genéricos permitem criar classes, interfaces e métodos que operam em tipos especificados pelo usuário
- Permitem reutilizar código para diferentes tipos de dados, mantendo a segurança de tipo em tempo de compilação
- Parâmetros de tipo são especificados na definição da classe e usados no forma de <T>

Reescrevendo classe Caixa para usar tipos genéricos

```
public class Caixa<T>{  
    private T conteudo;  
  
    public Caixa(T obj){  
        this.conteudo = obj;  
    }  
  
    public void set(T obj){  
        this.conteudo = obj;  
    }  
  
    public T getConteudo(){  
        return this.conteudo;  
    }  
}
```

```
public class App{  
  
    public static void main(String[] args){  
  
        Caixa<String> c = new Caixa<>("Olá mundo");  
        Caixa<Pessoa> d = new Caixa<>(new Pessoa("Joao"));  
  
        // não precisa de typecasting  
        String s = c.getConteudo();  
  
        // erro de compilação, tipos incompatíveis  
        String errado = d.getConteudo();  
    }  
}
```


Parâmetro de tipo limitado

- É possível restringir os tipos aceitos usando extends
 - Somente classes ou interfaces da hierarquia serão permitidas

```
public class Caixa<T extends Personagem>{  
    private T personagem;  
    .....  
}
```

```
public class App{  
    public static void main(String[] args){  
  
        // ok, pois Aldeao herda de Personagem  
        Caixa<Aldeao> c = new Caixa<>();  
  
        Caixa<String> n = new Caixa<>();// erro!  
    }  
}
```

Convenção de nomes em tipos genéricos

- Em Java, por convenção, parâmetros de tipo usam **letras maiúsculas**
- Nomes mais comuns:
 - T – *Type* (tipo genérico)
 - E – *Element* (elemento de coleção)
 - K – *Key* (chave)
 - V – *Value* (valor)
 - N – *Number* (tipos numéricos)
- Isso melhora a leitura e facilita identificar o papel de cada tipo

A API Collections do Java adota E para representar elementos (por exemplo, em `List<E>`)

Exercício: Classe Pessoa que permita comparações

- A classe `String` implementa a interface `Comparable<String>` e possui o método `compareTo`, usado para comparar e ordenar elementos em coleções (como `ArrayList`)

```
String s1 = "Ama";  
String s2 = "Ana";  
String s3 = "Ana";  
System.out.println(s1.compareTo(s2)); // -1, pois "Ama" vem antes de "Ana" na ordem alfabética  
System.out.println(s2.compareTo(s1)); // 1, pois "Ana" vem depois de "Ama" na ordem alfabética  
System.out.println(s2.compareTo(s3)); // 0, pois "Ana" é igual a "Ana"
```

- A classe `LocalDate` também implementa `Comparable`

```
LocalDate d1 = LocalDate.of(1990, 1, 1);  
LocalDate d2 = LocalDate.of(1980, 1, 1);  
System.out.println(d1.compareTo(d2)); // 10, pois d1 é posterior a d2
```

Exercício: Classe Pessoa que permita comparações

- Crie uma classe para representar uma pessoa com nome, sobrenome, data de nascimento
 - Implemente a interface `Comparable<Pessoa>` para permitir a ordenação de pessoas em uma coleção
 - Critério de ordenação: nome, depois sobrenome, depois data de nascimento
- Na classe App crie uma coleção de pessoas e ordene usando `Collections.sort`

```
ArrayList<Pessoa> pessoas = new ArrayList<>();  
pessoas.add(new Pessoa("Joao", "Silva", LocalDate.of(1990, 1, 1)));  
pessoas.add(new Pessoa("Ana", "Paula", LocalDate.of(1989, 1, 1)));  
pessoas.add(new Pessoa("Joao", "Santos", LocalDate.of(1991, 1, 1)));  
pessoas.add(new Pessoa("Joao", "Silva", LocalDate.of(1980, 1, 1)));  
  
pessoas.sort(null); // ordena usando o compareTo da classe Pessoa  
IO.println(pessoas);
```